ADGEO: segmentação distribuída de geleiras com gRPC e eleição de líder em anel

Welbert Junio Afonso de Almeida, Bruno Menezes Rodrigues Oliveira Vaz
Davi Cândido de Almeida, Gabriel Valedo Batista Silva
João Pedro Torres, Vitor Leite Setragni

[1]Instituto de Ciências Exatas e Informática – PUC Minas
Campus Coração Eucarístico – Belo Horizonte, MG – Brasil
Computação Distribuída – Prof. Luigi D. C. Soares – 2026.2 – Entrega 1

Abstract. *This work presents an ADGEO prototype for distributed glacier segmentation inference. A coordinator sends overlapping image tiles to peers through gRPC, stitches probabilities and computes global pixel metrics. A manually implemented Chang-Roberts ring election replaces a failed coordinator. Local tests with independent processes validate communication, reconstruction and the submission of new jobs to the elected peer. Job replication and recovery are reserved for the second delivery. Scientific accuracy with trained weights and real glacier data remains to be evaluated.*

Resumo. *Este trabalho apresenta um protótipo do ADGEO para inferência distribuída de segmentação de geleiras. Um coordenador envia tiles sobrepostos por gRPC, reúne probabilidades e calcula métricas globais. A eleição de Chang-Roberts em anel substitui o coordenador após uma falha. Testes com processos independentes verificam comunicação, reconstrução e novos jobs no eleito. Replicação e retomada de jobs pertencem à segunda entrega. A qualidade com pesos treinados e imagens reais permanece pendente de avaliação.*

1. Problema e escopo

O ADGEO identifica a área ocupada por geleiras em imagens de satélite. A proposta desta etapa é dividir o processamento de uma cena em unidades menores e executá-las em diferentes nós. A arquitetura utiliza RPC por gRPC e uma eleição de líder implementada na aplicação, atendendo às duas funcionalidades selecionadas para a primeira entrega.

A base de aprendizado já existente é o `UNet.ipynb`, com entrada de 13 bandas, classes fundo/geleira e pixels inválidos ignorados. A U-Net combina codificador, decodificador e conexões de passagem de características [Ronneberger et al. 2015]. O notebook realiza treinamento centralizado. Neste protótipo, os peers usam os mesmos pesos para **inferência**; não se sincronizam gradientes nem se treinam modelos independentes para depois juntar suas máscaras.

2. Arquitetura e mensagens

Cada peer executa o mesmo serviço Python e pode assumir os papéis de worker e coordenador. O cliente descobre o líder por consultas de estado e submete uma cena com identificador único, gabarito opcional e parâmetros de divisão. O líder distribui tiles em paralelo, recebe probabilidades e constrói a máscara final. Há uma inferência por vez em cada worker; o líder também pode trabalhar como worker.

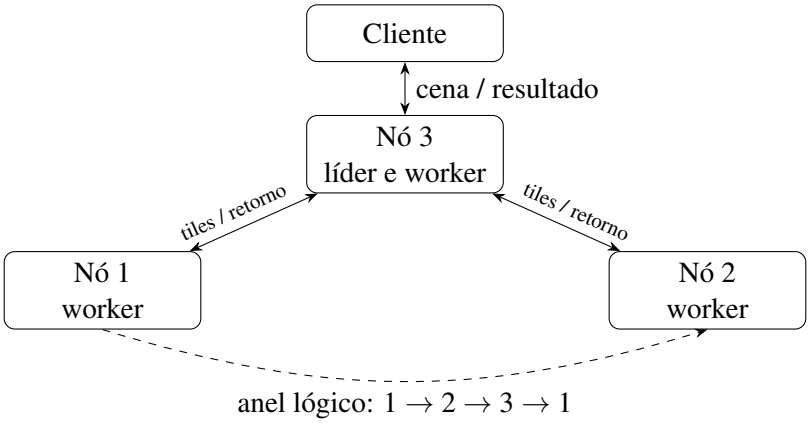


Figura 1. Fluxo de dados. Election e Coordinator percorrem o anel; Heartbeat consulta o estado dos peers. Os peers também participam da eleição em anel.

Tabela 1. Principais RPCs definidos em `cluster.proto`.

RPC	Comunicação	Conteúdo principal
SubmitJob / GetJob	cliente → líder	ID, cena, gabarito e parâmetros / estado e máscara
InferTile	líder → worker	IDs, posição, tensor, validade, hash e rodada; resposta com probabilidades
Heartbeat	cliente/peer → peer	ID local, líder conhecido, rodada e hash do modelo
Election / Coordinator	peer → sucessor	rodada e candidato / rodada e líder eleito

Os membros e seus endereços são configurados previamente. IDs positivos e únicos definem a ordem do anel. Quando um sucessor não responde, tenta-se o próximo membro configurado. O cliente comunica-se diretamente com o novo líder após a eleição. Cada nó mantém arquivos próprios; não se exige armazenamento compartilhado.

2.1. Divisão, costura e métricas

Cada tile recebe ID derivado da posição, coordenadas (x, y) e dimensões válidas. O tamanho padrão é 128×128, com sobreposição de 32 pixels e passo 96. A última posição é ajustada para cobrir a borda; cenas menores recebem padding. Na reconstrução, somam-se probabilidades e contagens por posição e aplica-se o limiar 0,5 à média. IDs já incorporados são ignorados. A média suaviza divergências na sobreposição, mas seu efeito na qualidade precisa ser medido.

3. Implementação dos requisitos

3.1. RPC e integração com a U-Net

O contrato Protobuf gera os stubs Python; o serviço e o cliente usam `grpc.aio`. A aplicação define prazos para as chamadas e repete tiles em outro worker quando há erro de comunicação. A definição de serviços e geração de stubs seguem a documentação do gRPC [gRPC 2026a]; prazos explícitos limitam a espera por respostas [gRPC 2026b].

As matrizes são transmitidas como bytes NPY, sem pickle. A aplicação valida dimensões, tipos, rótulos e tamanho antes de processá-las. Cada mensagem tem limite de 64 MiB e cada array de 48 MiB; o protótipo usa RPCs unários. O paralelismo é limitado por uma fila por worker e uma janela de tarefas, reduzindo o acúmulo de tiles na memória.

As classes da U-Net foram extraídas do notebook, mantendo nomes e formatos das camadas. O adaptador carrega a chave `modelo` do checkpoint, usa `eval()` e `inference_mode()` e obtém a probabilidade da classe geleira por softmax. A média e o desvio são os calculados no treino, nas mesmas 13 bandas. O hash dos pesos, da normalização e da ordem de bandas impede misturar versões de modelo no mesmo job. O dispositivo pode ser CPU ou CUDA; a verificação executada nesta sessão foi em CPU.

As métricas são calculadas na máscara final, contando cada pixel válido uma única vez. Para verdadeiros positivos TP, falsos positivos FP e falsos negativos FN:

$$\mathrm{IoU} = \frac{TP}{TP + FP + FN}, \qquad \mathrm{Dice} = \frac{2TP}{2TP + FP + FN}.$$

Não se faz média dos índices dos tiles sobrepostos. Denominadores nulos produzem valor ausente. Áreas em km^2 somente são informadas quando a área constante do pixel em m^2 é fornecida; sem esse dado, retornam-se contagens.

3.2. Eleição de líder em anel

A eleição implementa as regras de Chang-Roberts [Chang e Roberts 1979]: um nó inicia enviando seu ID ao sucessor. Ao receber um candidato maior, encaminha-o; um menor é substituído pelo ID local se o nó ainda não participa, ou descartado se já participa. Ao receber o próprio ID, o nó se elege e circula `Coordinator`. Em um anel estável, o pior caso usa $O(n^2)$ mensagens de eleição, mais $O(n)$ para anunciar o vencedor.

A aplicação acrescenta rodadas ordenadas por (contador persistido, ID iniciador), rejeitando mensagens antigas. Antes de iniciar uma rodada, consulta os contadores dos peers para permitir o retorno de um nó atrasado. Heartbeats utilizam tempo monotônico local. A ausência de resposta por um intervalo configurado gera suspeita e dispara nova eleição. O timeout não constitui prova de falha.

Os jobs desta etapa ficam em memória no coordenador atual. Se ele cair, a eleição permite receber novos trabalhos, mas não transfere os jobs anteriores. A entrada deve ser reenviada pelo cliente. A replicação e a recuperação automática são extensões da segunda etapa. Erros de comunicação com workers são limitados por deadlines e tentativas em outro worker.

4. Testes da primeira entrega

Foram aprovados 22 testes automatizados em 31/10/2026: divisão e costura, sobreposição, pixels inválidos, métricas, validação NPY, leitura NetCDF, execução com dois e três nós, idempotência no líder atual e eleição. A integração U-Net foi verificada em CPU com pesos artificiais e canais reduzidos nas mesmas classes do notebook, comparando a inferência local por tiles com gRPC.

A demonstração desta versão usou três processos e uma cena sintética de 13 bandas e 577×641 pixels, dividida em 42 tiles. O backend de teste devolve a primeira banda como probabilidade. As máscaras finais coincidiram com a referência local; os logs e o arquivo `summary.json` estão em `output/tests/demo`.

Tabela 2. Verificações da primeira etapa, sem replicação de jobs.

Verificação	Resultado
Processamento inicial	42 tiles; líder 3
Encerramento do líder 3	Nó 2 eleito em 2,23 s
Histórico do job anterior no eleito	Ausente, como previsto
Novo job enviado ao eleito	42 tiles; máscara correta

Esses resultados verificam a implementação distribuída. Não medem qualidade de segmentação de geleiras nem ganho de velocidade entre máquinas físicas. Pesos treinados, normalização e base real não foram fornecidos; o adaptador aceita esses arquivos quando disponíveis.

5. Desafios e responsabilidades

Foram tratados a dupla contagem nas sobreposições, a divergência de versões do modelo e as mensagens antigas de eleição. As soluções foram costurar antes de calcular métricas, identificar o modelo por hash e ordenar as rodadas. Caminhos longos de arquivos no Windows também receberam tratamento explícito. Timeout gera suspeita de falha; o protocolo não impede líderes diferentes em partições da rede.

Em cada dupla, teoria e implementação são compartilhadas. Bruno Menezes Rodrigues Oliveira Vaz e João Pedro Torres respondem por tiles, costura e métricas globais; Gabriel Valedo Batista Silva e Davi Cândido de Almeida, pelo contrato Protobuf, comunicação gRPC e integração U-Net; e Vitor Leite Setragni e Welbert Junio Afonso de Almeida, pela eleição em anel, heartbeat e testes de troca de coordenador.

Referências

Chang, E.; Roberts, R. (1979). An improved algorithm for decentralized extrema-finding in circular configurations of processes. *CACM*, 22(5), 281–283. doi:10.1145/359104.359108.

gRPC (2026). Basics tutorial: Python. Documentação oficial. Acesso em 31 out. 2026.

gRPC (2026). Deadlines. Documentação oficial. Acesso em 31 out. 2026.

Ronneberger, O.; Fischer, P.; Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*, 234–241. arXiv:1505.04597.